Pre-entrega 6 — Analítica avanzada in-stream con Amazon Redshift

Redshift Streaming Ingestion desde Kinesis, vistas materializadas de tiempo real e integración con el lakehouse Apache Iceberg

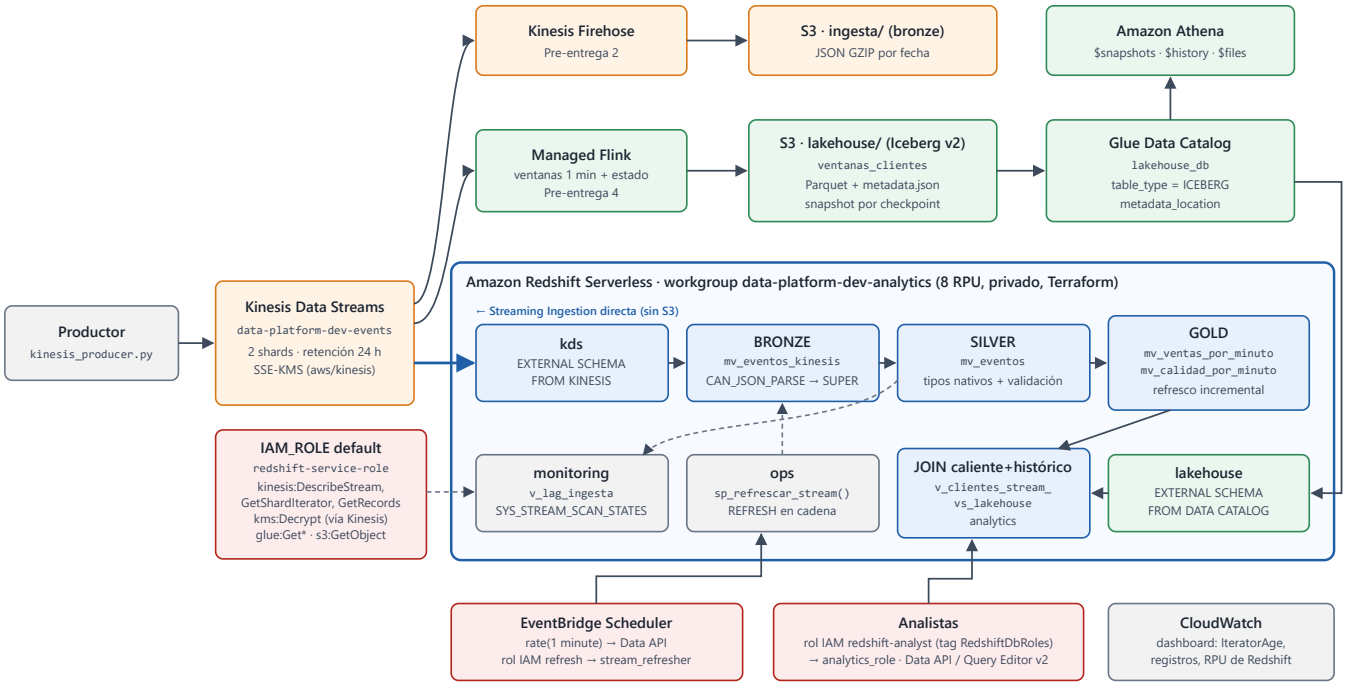
Autor	Hector Gabaldo
Fecha	10/11/2026
Cuenta / región	659500704179 · us-east-1
Repositorio	terraform-data-platform · rama `feature/lakehouse-iceberg`
Código	`infra/modules/redshift` · `sql/redshift_streaming.sql` · `scripts/redshift_sql.py`

1. Resumen

Se agregó a la plataforma una capa de consulta analítica de baja latencia con **Amazon Redshift Serverless**, desplegada con Terraform. Redshift lee el stream `data-platform-dev-events` con **Streaming Ingestion**, sin pasar por S3. Tres capas de vistas materializadas validan el JSON, lo convierten a tipos nativos y lo agregan por minuto. Un `EXTERNAL SCHEMA` sobre el Glue Data Catalog permite consultar con JOIN, en la misma sesión SQL, los datos calientes del stream y el histórico que Flink escribe en Apache Iceberg (Pre-entrega 5).

Criterio de evaluación	Resultado	Evidencia
Funcionalidad: datos de Kinesis a Redshift sin errores de casting	Cumplido	710 registros ingeridos (64 con schema drift), 0 errores en `sys_stream_scan_errors`. Los registros con *schema drift* se conservan y se clasifican, sin cortar la ingesta (§6.1, §6.2)
Integración: Iceberg y stream en la misma sesión SQL	Cumplido	Vista `analytics.v_clientes_stream_vs_lakehouse` y JOIN ad hoc; Redshift lee la tabla como `file_format = Iceberg` (§6.4, §6.5)
Seguridad: autenticación IAM entre servicios	Cumplido	Redshift→Kinesis/Glue/S3 con `IAM_ROLE default`; scheduler y analistas con credenciales IAM temporales; rol `analytics_role` sin acceso al dato crudo (§3, §6.6)
Claridad: SQL comentado y con buenas prácticas de nombrado	Cumplido	Un único script por secciones, con convenciones `mv_` / `v_` / `sp_` y sufijo `_ts` (§5)

2. Arquitectura actualizada



Líneas punteadas: el procedimiento de refresco actúa sobre la cadena bronze→silver→gold; monitoring lee silver y las vistas del sistema; el rol IAM autoriza a Redshift frente a Kinesis, KMS, Glue y S3.

Flujo

- **Transporte:** el productor escribe JSON en Kinesis. El mismo stream alimenta tres consumidores independientes: Firehose (archivo bronze en S3), Managed Flink (lakehouse Iceberg) y Redshift (analítica in-stream).
- **Ingesta directa:** `kds` mapea los streams de la cuenta como tablas. La vista bronze lee de cada shard los registros posteriores al último *sequence number* que materializó y descifra los datos a través de Kinesis con el rol IAM.
- **Modelado:** silver convierte el SUPER a tipos nativos y marca los registros que no cumplen el contrato. Gold agrega por minuto; las consultas leen totales ya calculados.
- **Lakehouse:** `lakehouse` apunta a `lakehouse_db` del Glue Data Catalog. Redshift resuelve el último snapshot desde `metadata_location` y lee los Parquet que indican los manifests.
- **Operación:** EventBridge Scheduler invoca cada minuto `CALL ops.sp_refrescar_stream()` por la Data API. El dashboard de CloudWatch y las vistas de `monitoring` miden el lag.

3. Infraestructura como código (Terraform)

Módulo nuevo `infra/modules/redshift`, invocado desde `infra/environments/dev/main.tf`. Para cumplir el requisito de Redshift Serverless de subredes en 3 zonas de disponibilidad, la VPC pasó a 3 AZ (`az_count = 3`). El cambio sólo agregó una subred y su tabla de rutas.

Recurso	Configuración relevante
`aws_redshiftserverless_namespace`	Base `analytics`. Contraseña del admin generada y rotada por Redshift en Secrets Manager (`manage_admin_password`): no aparece en el código ni en el state. Rol por defecto = `redshift-service-role`. Exporta los logs de usuarios, conexiones y actividad.

`aws_redshiftserverless_workgroup`	8 RPU base, `publicly_accessible = false`, subredes privadas, `require_ssl = true` y `max_query_execution_time = 900`
`aws_security_group`	Entrada 5439 sólo desde el CIDR de la VPC; salida sólo hacia la VPC
`aws_redshiftserverless_usage_limit`	Tope de 16 RPU-hora por día con `breach_action = deactivate` (control de costos)
`aws_iam_role.redshift_service`	Trust: `redshift.amazonaws.com` / `redshift-serverless.amazonaws.com` con `aws:SourceAccount`. Permisos: `kinesis:DescribeStream`, `DescribeStreamSummary`, `GetShardIterator`, `GetRecords` y `ListShards` sobre el ARN del stream; `kms:Decrypt` sólo `kms:ViaService = kinesis`; `glue:Get*` sobre `lakehouse_db`; `s3:GetObject` sobre `lakehouse/*`. Sólo lectura.
`aws_iam_role.analyst`	Tag `RedshiftDbRoles = analytics_role`. Sólo `redshift-serverless:GetCredentials` y `redshift-data:*Statement` sobre el workgroup; para ver resultados, condición `statement-owner-iam-userid = ${aws:userid}`.
`aws_iam_role.refresh` + `aws_scheduler_schedule`	EventBridge Scheduler, `rate(1 minute)`, destino `redshiftdata:executeStatement` sin secreto. Tag `RedshiftDbRoles = stream_refresher`. Se activa con `redshift_refresh_schedule_enabled`.
`aws_cloudwatch_dashboard`	IncomingRecords / GetRecords.Records, `GetRecords.IteratorAgeMilliseconds`, ComputeCapacity y ComputeSeconds de Redshift

Red: el workgroup no tiene endpoint público. Con `enhanced_vpc_routing = false`, Redshift llega a Kinesis, Glue y S3 por su red administrada, así que la VPC sin NAT no necesita endpoints de interfaz. Para operar se usa la Redshift Data API (HTTPS + IAM), que no requiere abrir puertos.

Recursos creados por `terraform apply`

```
module.redshift.aws_iam_role.analyst: Creation complete after 1s
module.redshift.aws_iam_role.redshift_service: Creation complete after 1s
module.redshift.aws_iam_role.refresh: Creation complete after 1s
module.network.aws_subnet.private["us-east-1c"]: Creation complete after 2s
module.redshift.aws_iam_role_policy.redshift_service: Creation complete after 1s
module.network.aws_route_table.private["us-east-1c"]: Creation complete after 1s
module.redshift.aws_redshiftserverless_namespace.this: Creation complete after 2s
module.redshift.aws_security_group.redshift: Creation complete after 4s
module.network.aws_route_table_association.private["us-east-1c"]: Creation complete after 1s
module.redshift.aws_vpc_security_group_ingress_rule.redshift_from_vpc: Creation complete after 0s
module.redshift.aws_redshiftserverless_workgroup.this: Creation complete after 1m9s
module.redshift.aws_redshiftserverless_usage_limit.daily_compute: Creation complete after 1s
module.redshift.aws_scheduler_schedule.refresh_stream_mv: Creation complete after 1s
module.redshift.aws_cloudwatch_dashboard.streaming_analytics: Creation complete after 1s
module.redshift.aws_iam_role_policy.analyst: Creation complete after 1s
module.redshift.aws_iam_role_policy.refresh: Creation complete after 1s
module.redshift.aws_vpc_security_group_egress_rule.redshift_to_vpc: Creation complete after 1s
Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
```

La primera ejecución creó 16 de los 17 recursos y falló en la regla de salida del security group: la descripción tenía un acento ("sólo") y EC2 sólo admite ASCII. Se corrigió el texto y un segundo `apply` creó la regla (*1 added*; el *1 changed* es la deriva previa de `run_configuration` de Flink, que no arranca la aplicación). Salida completa: `docs/evidencias-redshift/terraform_apply.txt`.

4. Diseño de la capa SQL

Objeto	Capa	Qué hace

`kds."data-platform-dev-events"`	Mapeo	Stream de Kinesis expuesto como tabla (`kinesis_data` VARBYTE + metadatos del registro)
`stream.mv_eventos_kinesis`	Bronze (streaming)	Valida el JSON con `CAN_JSON_PARSE`, lo guarda como SUPER con `JSON_PARSE` y conserva `approximate_arrival_timestamp`, `refresh_time`, shard y sequence number
`stream.mv_eventos`	Silver	`event_id` BIGINT, `event_type` / `cliente_id` VARCHAR, `importe` DECIMAL(12,2), `event_ts` TIMESTAMP (UTC), `es_valido` y `motivo_invalido`
`analytics.mv_ventas_por_minuto`	Gold	Operaciones, compras, importe y compra máxima por minuto y cliente
`analytics.mv_calidad_por_minuto`	Gold	Registros válidos y errores por motivo y por minuto: el "errores por minuto" del stream
`analytics.v_clientes_stream_vs_lakehouse`	Servicio	JOIN de los últimos 15 minutos del stream con el histórico Iceberg de cada cliente
`monitoring.v_lag_ingesta`, `v_scans_stream`, `v_refrescos`	Monitoreo	Lag por minuto, lectura por shard y tipo y duración de cada refresco
`ops.sp_refrescar_stream()`	Operación	Refresca la cadena en orden (`SECURITY DEFINER`)

Decisiones tomadas a partir de pruebas en el workgroup

- **Validar antes de extraer (schema drift).** Si la vista de streaming fallara con un registro que no es JSON, se detendría toda la ingesta. Bronze sólo aplica `CAN_JSON_PARSE` y `JSON_PARSE`, que nunca fallan: el registro inválido queda con `is_valid_json = false`. Una vista de streaming tampoco admite alias de subconsulta ni navegar `JSON_PARSE(...).campo` (error de sintaxis), así que la extracción tipada está en silver.
- **Conversión "lax" desde SUPER, sin errores de casting.** `payload.importe::DECIMAL` con `"N/A"` devuelve NULL en lugar de abortar. Además, `JSON_TYPEOF` exige el tipo JSON correcto antes de convertir.
- **Funciones inmutables, para que el refresco sea incremental.** Convertir VARCHAR→TIMESTAMP, `TO_TIMESTAMP`, `EXTRACT(EPOCH)` o `DATE_TRUNC` sobre TIMESTAMPTZ son funciones "mutables" (dependen de la sesión) y obligan a recalcular la vista completa en cada refresco (`svv_mv_info.state = 0`). La conversión SUPER→TIMESTAMP sí es inmutable, pero conserva la hora de reloj e ignora el offset. Por eso el contrato exige timestamps en UTC (`+00:00` / `Z`) y cualquier otro offset se marca `timestamp_no_utc`, en vez de guardar una hora incorrecta. Resultado: las 4 vistas tienen `state = 1`.
- **Refresco orquestado, no AUTO REFRESH.** Redshift no admite AUTO REFRESH en vistas definidas sobre otras vistas materializadas. Además, sobre un stream no permite fijar la cadencia y en Serverless mantendría el cómputo activo de forma continua. El procedimiento hace `COMMIT` entre capas: dentro de una misma transacción, silver veía a bronze como "no actualizada" y no incorporaba las filas nuevas.
- **Pre-agregar antes del JOIN.** Un cliente tiene muchos minutos en el stream y muchas ventanas en Iceberg. Unir esas filas sin agregar multiplica las sumas, así que cada lado se agrega primero.

5. Código SQL: Streaming Ingestion y vistas materializadas

Archivo `sql/redshift_streaming.sql`, que consolida todo el SQL. Se ejecuta con `python scripts/redshift_sql.py --secret-arn <arn> --file sql/redshift_streaming.sql`, que usa la Redshift Data API y separa el archivo por secciones.

```
/* ============================================================================
   Pre-entrega 6 — Analítica avanzada in-stream con Amazon Redshift
   ----------------------------------------------------------------------------
```

```
   Kinesis Data Streams ──(Streaming Ingestion, sin S3)──> Redshift Serverless
   Glue Data Catalog (Iceberg) ──(EXTERNAL SCHEMA)──────> Redshift Serverless

   Destino   : workgroup data-platform-dev-analytics, base "analytics"
               (infra/modules/redshift, desplegado con Terraform)
   Ejecución : python scripts/redshift_sql.py --secret-arn <redshift_admin_secret_arn>
                       --file sql/redshift_streaming.sql
               El runner usa la Redshift Data API (HTTPS + IAM) y separa el
               archivo por secciones "-- @seccion:". Las secciones "reset" y
               "validacion-analista" sólo se ejecutan si se piden con --seccion.

   Capas (medallion dentro de Redshift):
     kds.<stream>                    objeto externo: mapeo del stream de Kinesis
     stream.mv_eventos_kinesis       BRONZE  vista materializada de streaming:
                                             valida el JSON y lo guarda como SUPER
     stream.mv_eventos               SILVER  campos tipados + validación de contrato
     analytics.mv_ventas_por_minuto  GOLD    ventas por minuto y cliente
     analytics.mv_calidad_por_minuto GOLD    registros válidos / con errores por minuto
     analytics.v_clientes_stream_vs_lakehouse   JOIN stream (caliente) + Iceberg (histórico)
     monitoring.*                    lag de ingesta y estado de los refrescos
     ops.sp_refrescar_stream()       refresco incremental de toda la cadena

   Convenciones de nombres: mv_ = vista materializada, v_ = vista, sp_ =
   procedimiento, sufijo _ts = TIMESTAMP en UTC. Identificadores en minúscula.
   ============================================================================ */


-- @seccion: seguridad
-- ----------------------------------------------------------------------------
-- 1. Roles de base de datos (RBAC)
--    analytics_role   : analistas. Sólo lectura de la capa analytics, del
--                       lakehouse y de la vista de lag. No ve el dato crudo.
--    stream_refresher : sólo puede ejecutar el procedimiento de refresco.
--    Ambos se otorgan automáticamente a los roles IAM que tienen el tag
--    RedshiftDbRoles (data-platform-dev-redshift-analyst / -refresh) cuando se
--    conectan con credenciales temporales: no hay usuarios con contraseña.
-- ----------------------------------------------------------------------------
CREATE ROLE analytics_role;
CREATE ROLE stream_refresher;

-- Nadie crea objetos en el esquema public por defecto.
REVOKE CREATE ON SCHEMA public FROM PUBLIC;


-- @seccion: esquemas-externos
-- ----------------------------------------------------------------------------
-- 2. Mapeo del stream de Kinesis (Streaming Ingestion)
--    IAM_ROLE default = data-platform-dev-redshift-service-role, el rol por
--    defecto del namespace (Terraform). Redshift lo asume para llamar a
--    kinesis:DescribeStream / GetShardIterator / GetRecords: autenticación
--    IAM entre servicios, sin claves de acceso.
--    Cada stream de la cuenta aparece como una tabla de este esquema.
-- ----------------------------------------------------------------------------
CREATE EXTERNAL SCHEMA IF NOT EXISTS kds
FROM KINESIS
IAM_ROLE default;

-- ----------------------------------------------------------------------------
-- 3. Lakehouse: Glue Data Catalog de la Pre-entrega 5 (tablas Apache Iceberg)
--    Redshift lee la metadata de Iceberg (metadata_location en Glue) y los
--    Parquet de S3 con el mismo rol. Es sólo lectura: los commits los hace Flink.
-- ----------------------------------------------------------------------------
CREATE EXTERNAL SCHEMA IF NOT EXISTS lakehouse
FROM DATA CATALOG
DATABASE 'lakehouse_db'
IAM_ROLE default
REGION 'us-east-1';


-- @seccion: esquemas-locales
-- ----------------------------------------------------------------------------
-- 4. Esquemas locales: separan el dato crudo (stream) de lo que consumen los
--    analistas (analytics). Los permisos se dan por esquema en la sección 10.
-- ----------------------------------------------------------------------------
CREATE SCHEMA IF NOT EXISTS stream;      -- bronze y silver (uso interno)
```

```
CREATE SCHEMA IF NOT EXISTS analytics;   -- gold y vistas de negocio
CREATE SCHEMA IF NOT EXISTS monitoring;  -- lag y salud de la ingesta
CREATE SCHEMA IF NOT EXISTS ops;         -- procedimientos de operación


-- @seccion: ingesta
-- ----------------------------------------------------------------------------
-- 5. BRONZE — vista materializada de streaming sobre el stream de Kinesis
--
--    * Cada REFRESH lee sólo los registros nuevos de cada shard desde el último
--      sequence number confirmado (refresco incremental). El primer REFRESH
--      lee desde TRIM_HORIZON (retención del stream: 24 h).
--    * Validación ANTES de extraer (schema drift): CAN_JSON_PARSE evita que un
--      registro que no es JSON haga fallar el refresco. El registro se conserva
--      con is_valid_json = false para poder auditarlo, en lugar de perderse.
--    * JSON_PARSE convierte el JSON en SUPER (semiestructurado): si mañana el
--      productor agrega un campo, la ingesta no cambia.
--    * Una vista de streaming no admite alias de subconsulta ni navegar el
--      resultado de JSON_PARSE en la misma expresión: la extracción tipada se
--      hace en la capa silver, sobre la columna SUPER ya materializada.
--    * AUTO REFRESH NO: la cadencia la define ops.sp_refrescar_stream() vía
--      EventBridge Scheduler (cada 1 minuto). Ver sección 9.
-- ----------------------------------------------------------------------------
CREATE MATERIALIZED VIEW stream.mv_eventos_kinesis
SORTKEY (llegada_kinesis_ts)
AUTO REFRESH NO
AS
SELECT
    approximate_arrival_timestamp                    AS llegada_kinesis_ts,  -- cuándo lo aceptó Kinesis (UTC)
    refresh_time                                     AS ingesta_redshift_ts, -- cuándo lo materializó Redshift (UTC)
    partition_key,
    shard_id,
    sequence_number,
    CAN_JSON_PARSE(kinesis_data)                     AS is_valid_json,
    CASE WHEN CAN_JSON_PARSE(kinesis_data)
         THEN JSON_PARSE(kinesis_data)
    END                                              AS payload              -- SUPER
FROM kds."data-platform-dev-events";


-- @seccion: modelado
-- ----------------------------------------------------------------------------
-- 6. SILVER — campos JSON convertidos a tipos nativos de Redshift
--
--    Contrato del evento (scripts/kinesis_producer.py):
--      {"event_id": 1, "event_type": "compra", "cliente_id": "cliente-001",
--       "importe": 123.45, "timestamp": "2026-11-10T16:16:25.281662+00:00"}
--
--    * Las conversiones desde SUPER son "lax": si el tipo no coincide
--      (importe = "N/A", timestamp = "10/11/2026 25:61") devuelven NULL en
--      lugar de abortar el refresco. Por eso no hay errores de casting.
--    * JSON_TYPEOF valida el tipo JSON antes de convertir (un event_id "12"
--      como texto no se acepta como número).
--    * event_ts: se exige ISO-8601 con zona UTC (+00:00 o Z). La conversión
--      SUPER -> TIMESTAMP conserva la hora de reloj, así que un offset distinto
--      daría una hora incorrecta: esos registros se marcan inválidos.
--      (Convertir VARCHAR -> TIMESTAMP o usar TIMESTAMPTZ con DATE_TRUNC son
--      funciones "mutables" y obligarían a recalcular la vista completa.)
--    * es_valido / motivo_invalido: el registro se conserva y se clasifica;
--      las capas gold filtran por es_valido y miden los errores.
-- ----------------------------------------------------------------------------
CREATE MATERIALIZED VIEW stream.mv_eventos
SORTKEY (event_ts)
AUTO REFRESH NO
AS
SELECT
    shard_id,
    sequence_number,
    llegada_kinesis_ts,
    ingesta_redshift_ts,

    CASE WHEN JSON_TYPEOF(payload.event_id) = 'number'
         THEN payload.event_id::BIGINT END                          AS event_id,
    CASE WHEN JSON_TYPEOF(payload.event_type) = 'string'
         THEN LOWER(payload.event_type::VARCHAR(32)) END            AS event_type,
```

```
    CASE WHEN JSON_TYPEOF(payload.cliente_id) = 'string'
         THEN payload.cliente_id::VARCHAR(64) END                    AS cliente_id,
    CASE WHEN JSON_TYPEOF(payload.importe) = 'number'
         THEN payload.importe::DECIMAL(12, 2) END                    AS importe,
    CASE WHEN JSON_TYPEOF(payload."timestamp") = 'string'
          AND (payload."timestamp"::VARCHAR(64) LIKE '%+00:00'
               OR payload."timestamp"::VARCHAR(64) LIKE '%Z')
         THEN payload."timestamp"::TIMESTAMP END                     AS event_ts,

    -- Registro utilizable por las métricas: JSON válido y los 5 campos con tipo correcto.
    -- COALESCE: un campo ausente da NULL en la comparación; se trata como inválido.
    COALESCE(is_valid_json
     AND JSON_TYPEOF(payload.event_id) = 'number'
     AND JSON_TYPEOF(payload.event_type) = 'string'
     AND JSON_TYPEOF(payload.cliente_id) = 'string'
     AND JSON_TYPEOF(payload.importe) = 'number'
     AND payload."timestamp"::TIMESTAMP IS NOT NULL
     AND (payload."timestamp"::VARCHAR(64) LIKE '%+00:00'
          OR payload."timestamp"::VARCHAR(64) LIKE '%Z'), FALSE)     AS es_valido,

    CASE
        WHEN NOT is_valid_json                                  THEN 'json_invalido'
        WHEN JSON_TYPEOF(payload.event_id) IS DISTINCT FROM 'number'
          OR JSON_TYPEOF(payload.cliente_id) IS DISTINCT FROM 'string'
          OR JSON_TYPEOF(payload.event_type) IS DISTINCT FROM 'string' THEN 'campo_obligatorio_faltante'
        WHEN JSON_TYPEOF(payload.importe) IS DISTINCT FROM 'number' THEN 'importe_no_numerico'
        WHEN payload."timestamp"::TIMESTAMP IS NULL             THEN 'timestamp_invalido'
        WHEN NOT (payload."timestamp"::VARCHAR(64) LIKE '%+00:00'
                  OR payload."timestamp"::VARCHAR(64) LIKE '%Z')   THEN 'timestamp_no_utc'
    END                                                          AS motivo_invalido,

    payload                                                      -- SUPER original, para auditoría
FROM stream.mv_eventos_kinesis;


-- @seccion: agregaciones
-- ------------------------------------------------------------------------------
-- 7. GOLD — métricas de tiempo real (respuesta instantánea: ya están agregadas)
--    Sólo SUM/COUNT/MAX sobre columnas TIMESTAMP: Redshift las mantiene con
--    refresco incremental (svv_mv_info.state = 1), sumando sólo lo nuevo.
-- ------------------------------------------------------------------------------

-- Ventas por minuto (event time) y cliente.
CREATE MATERIALIZED VIEW analytics.mv_ventas_por_minuto
SORTKEY (minuto)
AUTO REFRESH NO
AS
SELECT
    DATE_TRUNC('minute', event_ts)                               AS minuto,
    cliente_id,
    COUNT(*)                                                     AS operaciones,
    SUM(CASE WHEN event_type = 'compra' THEN 1 ELSE 0 END)       AS compras,
    SUM(CASE WHEN event_type = 'compra' THEN importe ELSE 0 END) AS importe_compras,
    MAX(CASE WHEN event_type = 'compra' THEN importe END)        AS compra_maxima,
    MAX(llegada_kinesis_ts)                                      AS ultima_llegada_ts
FROM stream.mv_eventos
WHERE es_valido
GROUP BY 1, 2;

-- Calidad del stream: registros válidos y errores por minuto de llegada a Kinesis
-- (mide el schema drift: si aparece un motivo nuevo, el productor cambió).
CREATE MATERIALIZED VIEW analytics.mv_calidad_por_minuto
SORTKEY (minuto)
AUTO REFRESH NO
AS
SELECT
    DATE_TRUNC('minute', llegada_kinesis_ts)           AS minuto,
    COALESCE(motivo_invalido, 'ok')                    AS resultado,
    COUNT(*)                                           AS registros
FROM stream.mv_eventos
GROUP BY 1, 2;


-- @seccion: lakehouse
-- ------------------------------------------------------------------------------
```

```
-- 8. Integración Lakehouse: datos calientes (stream) + históricos (Iceberg)
--
--    lakehouse.ventanas_clientes es la tabla Iceberg que escribe Flink
--    (Pre-entrega 5): una fila por cliente y ventana de 1 minuto, con el
--    estado acumulado del cliente y alertas.
--    La vista compara el ritmo de compras de los últimos 15 minutos contra el
--    promedio histórico por ventana del mismo cliente.
--    WITH NO SCHEMA BINDING es obligatorio para vistas sobre tablas externas.
-- ------------------------------------------------------------------------------
CREATE OR REPLACE VIEW analytics.v_clientes_stream_vs_lakehouse AS
WITH caliente AS (          -- Redshift: vista materializada sobre Kinesis
    SELECT
        cliente_id,
        COUNT(*)                                AS minutos_activos,
        SUM(compras)                            AS compras_15m,
        SUM(importe_compras)                    AS importe_15m,
        (SUM(importe_compras) / COUNT(*))::DECIMAL(12, 2) AS importe_por_minuto,
        MAX(minuto)                             AS ultimo_minuto
    FROM analytics.mv_ventas_por_minuto
    WHERE minuto >= DATEADD(minute, -15, DATE_TRUNC('minute', GETDATE()))
    GROUP BY cliente_id
),
historico AS (              -- Iceberg en S3 vía Glue Data Catalog
    SELECT
        cliente_id,
        COUNT(*)                                AS ventanas_historicas,
        AVG(importe_total_compras)::DECIMAL(12, 2) AS importe_promedio_ventana,
        MAX(importe_acumulado)                  AS importe_acumulado_historico,
        SUM(CASE WHEN alerta = 'PICO_COMPRAS' THEN 1 ELSE 0 END) AS alertas_historicas,
        MAX(window_end)                         AS ultima_ventana_historica
    FROM lakehouse.ventanas_clientes
    GROUP BY cliente_id
)
SELECT
    c.cliente_id,
    c.minutos_activos,
    c.compras_15m,
    c.importe_15m,
    c.importe_por_minuto,
    h.ventanas_historicas,
    h.importe_promedio_ventana,
    ROUND(c.importe_por_minuto / NULLIF(h.importe_promedio_ventana, 0), 2) AS ratio_vs_historico,
    h.alertas_historicas,
    h.ultima_ventana_historica,
    c.ultimo_minuto,
    CASE
        WHEN h.cliente_id IS NULL                                       THEN 'CLIENTE_NUEVO'
        WHEN c.importe_por_minuto = 0 AND h.importe_promedio_ventana = 0 THEN 'SIN_COMPRAS'
        WHEN c.importe_por_minuto > 2 * h.importe_promedio_ventana      THEN 'POR_ENCIMA_DEL_HISTORICO'
        WHEN c.importe_por_minuto < 0.5 * h.importe_promedio_ventana    THEN 'POR_DEBAJO_DEL_HISTORICO'
        ELSE 'NORMAL'
    END                                                                 AS diagnostico
FROM caliente c
LEFT JOIN historico h ON h.cliente_id = c.cliente_id
WITH NO SCHEMA BINDING;


-- @seccion: monitoreo
-- ------------------------------------------------------------------------------
-- 9a. Lag de ingesta por minuto
--     lag_kinesis_a_redshift : desde que Kinesis aceptó el registro hasta que
--                              un REFRESH lo materializó (lo controla la cadencia).
--     lag_evento_a_kinesis   : desde que se generó el evento hasta Kinesis (red / productor).
-- ------------------------------------------------------------------------------
CREATE OR REPLACE VIEW monitoring.v_lag_ingesta AS
SELECT
    DATE_TRUNC('minute', llegada_kinesis_ts)                                        AS minuto,
    COUNT(*)                                                                        AS registros,
    ROUND(AVG(DATEDIFF(millisecond, llegada_kinesis_ts, ingesta_redshift_ts)) / 1000.0, 1) AS
lag_kinesis_a_redshift_prom_s,
    ROUND(MAX(DATEDIFF(millisecond, llegada_kinesis_ts, ingesta_redshift_ts)) / 1000.0, 1) AS
lag_kinesis_a_redshift_max_s,
    ROUND(AVG(DATEDIFF(millisecond, event_ts, llegada_kinesis_ts)) / 1000.0, 1)    AS lag_evento_a_kinesis_prom_s,
    MAX(ingesta_redshift_ts)                                                        AS ultimo_refresco_ts
FROM stream.mv_eventos
```

```
GROUP BY 1;

-- 9b. Lo que leyó cada REFRESH de cada shard (vista del sistema de streaming).
--     lag_from_latest = registros del shard todavía no ingeridos al terminar el scan.
CREATE OR REPLACE VIEW monitoring.v_scans_stream AS
SELECT
    record_time                AS scan_ts,
    mv_name,
    partition_id               AS shard_id,
    scanned_rows,
    skipped_rows,
    scanned_bytes,
    lag_from_latest,
    min_latency_s,
    max_latency_s,
    stream_record_time_min,
    stream_record_time_max
FROM sys_stream_scan_states
WHERE stream_name = 'data-platform-dev-events';

-- 9c. Historial de refrescos (tipo incremental / recálculo completo, duración).
CREATE OR REPLACE VIEW monitoring.v_refrescos AS
SELECT
    schema_name || '.' || mv_name                    AS vista,
    start_time,
    DATEDIFF(millisecond, start_time, end_time)      AS duracion_ms,
    status,
    refresh_type
FROM sys_mv_refresh_history
WHERE schema_name IN ('stream', 'analytics');


-- @seccion: refresco
-- ------------------------------------------------------------------------------
-- 10. Estrategia de refresco incremental
--
--     * Las vistas anidadas (silver/gold sobre bronze) no admiten AUTO REFRESH
--       y, en Serverless, AUTO REFRESH sobre el stream mantendría el cómputo
--       activo todo el tiempo. Se refresca la cadena completa, en orden, desde
--       este procedimiento.
--     * Cadencia: EventBridge Scheduler lo invoca cada 1 minuto (Terraform:
--       redshift_refresh_schedule_enabled / refresh_schedule_expression). Es el
--       equilibrio para métricas "por minuto": refrescar cada segundo saturaría
--       el workgroup y no mejora una métrica que se agrega por minuto.
--     * Equivale a REFRESH ... CASCADE sobre cada vista gold, pero refresca
--       bronze y silver una sola vez para las dos.
--     * Cada REFRESH es incremental: bronze lee sólo los sequence numbers nuevos
--       de cada shard; silver y gold procesan sólo esas filas.
--     * SECURITY DEFINER: corre con los permisos del dueño. Quien lo invoca
--       (rol stream_refresher) no necesita acceso a las tablas.
-- ------------------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE ops.sp_refrescar_stream()
AS $$
BEGIN
    -- COMMIT entre capas: dentro de una misma transacción la vista siguiente
    -- ve a la anterior como "no actualizada" y no incorpora las filas nuevas.
    REFRESH MATERIALIZED VIEW stream.mv_eventos_kinesis;       -- bronze: lee Kinesis
    COMMIT;
    REFRESH MATERIALIZED VIEW stream.mv_eventos;               -- silver
    COMMIT;
    REFRESH MATERIALIZED VIEW analytics.mv_ventas_por_minuto;  -- gold
    REFRESH MATERIALIZED VIEW analytics.mv_calidad_por_minuto; -- gold
    COMMIT;
END;
$$ LANGUAGE plpgsql
SECURITY DEFINER;


-- @seccion: permisos
-- ------------------------------------------------------------------------------
-- 11. Acceso restringido por rol (mínimo privilegio)
--     analytics_role : USAGE + SELECT sobre analytics, USAGE sobre lakehouse y
--                      sólo la vista de lag de monitoring. Sin acceso a stream
--                      (payload crudo) ni a kds (lectura directa del stream).
--     stream_refresher: sólo EXECUTE del procedimiento de refresco.
```

```
-- ------------------------------------------------------------------------------
REVOKE ALL ON SCHEMA stream     FROM PUBLIC;
REVOKE ALL ON SCHEMA analytics  FROM PUBLIC;
REVOKE ALL ON SCHEMA monitoring FROM PUBLIC;
REVOKE ALL ON SCHEMA ops        FROM PUBLIC;
REVOKE ALL ON SCHEMA kds        FROM PUBLIC;
REVOKE ALL ON SCHEMA lakehouse  FROM PUBLIC;
REVOKE ALL ON PROCEDURE ops.sp_refrescar_stream() FROM PUBLIC;

GRANT USAGE  ON SCHEMA analytics TO ROLE analytics_role;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics TO ROLE analytics_role;
-- Objetos que se creen más adelante en analytics heredan el permiso.
ALTER DEFAULT PRIVILEGES IN SCHEMA analytics GRANT SELECT ON TABLES TO ROLE analytics_role;

GRANT USAGE  ON SCHEMA lakehouse TO ROLE analytics_role;

GRANT USAGE  ON SCHEMA monitoring TO ROLE analytics_role;
GRANT SELECT ON monitoring.v_lag_ingesta TO ROLE analytics_role;

GRANT USAGE   ON SCHEMA ops TO ROLE stream_refresher;
GRANT EXECUTE ON PROCEDURE ops.sp_refrescar_stream() TO ROLE stream_refresher;

-- Primera carga: lee el stream desde TRIM_HORIZON y llena toda la cadena.
CALL ops.sp_refrescar_stream();


-- @seccion: validacion
-- ------------------------------------------------------------------------------
-- 12. Validación (ejecutar después de enviar eventos con kinesis_producer.py)
-- ------------------------------------------------------------------------------

-- V1. Refresco manual de la cadena (lo mismo que hace el scheduler).
CALL ops.sp_refrescar_stream();

-- V2. Bronze: registros ingeridos desde Kinesis, por shard y validez del JSON.
SELECT shard_id, is_valid_json, COUNT(*) AS registros,
       MIN(llegada_kinesis_ts) AS primera_llegada, MAX(llegada_kinesis_ts) AS ultima_llegada
FROM stream.mv_eventos_kinesis
GROUP BY 1, 2
ORDER BY 1, 2;

-- V3. Silver: campos JSON convertidos a tipos nativos (sin errores de casting).
SELECT event_id, event_type, cliente_id, importe, event_ts, llegada_kinesis_ts, ingesta_redshift_ts
FROM stream.mv_eventos
WHERE es_valido
ORDER BY llegada_kinesis_ts DESC, event_id DESC
LIMIT 10;

-- V4. Tipos de las columnas tipadas.
SELECT column_name, data_type, character_maximum_length, numeric_precision, numeric_scale
FROM svv_columns
WHERE table_schema = 'stream' AND table_name = 'mv_eventos'
ORDER BY ordinal_position;

-- V5. Schema drift: registros conservados y clasificados (no rompen la ingesta).
SELECT motivo_invalido, COUNT(*) AS registros,
       MIN(JSON_SERIALIZE(payload)) AS ejemplo
FROM stream.mv_eventos
WHERE NOT es_valido
GROUP BY 1
ORDER BY 2 DESC;

-- V6. Gold: ventas por minuto (últimos 10 minutos con datos).
SELECT minuto, SUM(operaciones) AS operaciones, SUM(compras) AS compras,
       SUM(importe_compras) AS importe_compras, MAX(compra_maxima) AS compra_maxima
FROM analytics.mv_ventas_por_minuto
GROUP BY 1
ORDER BY 1 DESC
LIMIT 10;

-- V7. Gold: calidad del stream por minuto.
SELECT minuto, resultado, registros
FROM analytics.mv_calidad_por_minuto
ORDER BY minuto DESC, registros DESC
LIMIT 15;
```

```
-- V8. JOIN stream (Redshift) + histórico (Iceberg) en la misma sesión.
SELECT *
FROM analytics.v_clientes_stream_vs_lakehouse
ORDER BY cliente_id;

-- V9. JOIN ad hoc: importe de la última hora del stream + estado acumulado que
--     Flink dejó en Iceberg. Cada lado se agrega ANTES del JOIN (un cliente
--     tiene muchos minutos y muchas ventanas: unirlos sin agregar duplicaría sumas).
SELECT s.cliente_id,
       s.compras                                      AS compras_stream_ultima_hora,
       s.importe_compras                              AS importe_stream_ultima_hora,
       i.ventanas_procesadas                          AS ventanas_flink,
       i.importe_acumulado::DECIMAL(12, 2)            AS importe_acumulado_iceberg,
       (i.importe_acumulado + s.importe_compras)::DECIMAL(12, 2) AS importe_total_cliente,
       i.ultima_ventana                               AS ultima_ventana_iceberg
FROM (SELECT cliente_id, SUM(compras) AS compras, SUM(importe_compras) AS importe_compras
      FROM analytics.mv_ventas_por_minuto
      WHERE minuto >= DATEADD(hour, -1, GETDATE())
      GROUP BY cliente_id) AS s
JOIN (SELECT cliente_id, MAX(ventanas_procesadas) AS ventanas_procesadas,
             MAX(importe_acumulado) AS importe_acumulado, MAX(window_end) AS ultima_ventana
      FROM lakehouse.ventanas_clientes
      GROUP BY cliente_id) AS i
  ON i.cliente_id = s.cliente_id
ORDER BY s.cliente_id;

-- V10. Metadatos de la tabla Iceberg vistos desde Redshift (Glue Data Catalog).
--      svv_external_tables trunca "parameters" a 128 caracteres; el historial
--      completo de snapshots se consulta en Athena ("ventanas_clientes$snapshots").
SELECT schemaname, tablename, location,
       TIMESTAMP 'epoch' + (REGEXP_SUBSTR(parameters, '"iceberg.table.lastUpdatedMs":"([0-9]+)"', 1, 1, 'e')::BIGINT /
1000)
                           * INTERVAL '1 second'                          AS ultimo_commit_iceberg,
       parameters                                                         AS parametros_glue
FROM svv_external_tables
WHERE schemaname = 'lakehouse';

SELECT columnnum, columnname, external_type, part_key
FROM svv_external_columns
WHERE schemaname = 'lakehouse' AND tablename = 'ventanas_clientes'
ORDER BY columnnum;

-- Cómo leyó Redshift la tabla: file_format = Iceberg y archivos resueltos a
-- partir de los manifests (no lista S3 ni usa particiones de Glue).
SELECT start_time, TRIM(table_name) AS tabla, TRIM(file_format) AS formato,
       scanned_files, returned_rows, returned_bytes, duration AS duracion_us
FROM sys_external_query_detail
WHERE file_format = 'Iceberg'
ORDER BY start_time DESC
LIMIT 5;

-- V11. Lag de ingesta (minutos recientes).
SELECT *
FROM monitoring.v_lag_ingesta
ORDER BY minuto DESC
LIMIT 10;

-- V12. Scans de Kinesis por shard (últimos refrescos).
SELECT *
FROM monitoring.v_scans_stream
ORDER BY scan_ts DESC
LIMIT 8;

-- V13. Refrescos: deben ser incrementales.
SELECT *
FROM monitoring.v_refrescos
ORDER BY start_time DESC
LIMIT 8;

-- V14. Estado de las vistas: state = 1 -> admite refresco incremental.
SELECT schema_name, name, autorefresh, is_stale, state
FROM svv_mv_info
WHERE schema_name IN ('stream', 'analytics')
ORDER BY schema_name, name;
```

```
-- V15. Errores de lectura del stream (registros > 1 MB, etc.): debe estar vacío.
SELECT record_time, mv_name, partition_id, error_code, error_reason
FROM sys_stream_scan_errors
ORDER BY record_time DESC
LIMIT 10;

-- V16. Refresco programado: lo ejecuta el usuario IAMR del rol IAM del scheduler,
--      cada ~60 s, sin contraseña.
SELECT TRIM(u.usename)                          AS usuario_bd,
       TRIM(q.query_text)                       AS sentencia,
       TRIM(q.status)                           AS estado,
       q.start_time,
       DATEDIFF(millisecond, q.start_time, q.end_time) AS duracion_ms
FROM sys_query_history q
JOIN pg_user u ON u.usesysid = q.user_id
WHERE q.query_text ILIKE 'CALL ops.sp_refrescar_stream%'
ORDER BY q.start_time DESC
LIMIT 6;


-- @seccion: validacion-analista
-- ------------------------------------------------------------------------------
-- 13. Validación de permisos. Ejecutar asumiendo el rol IAM de analistas:
--       python scripts/redshift_sql.py --profile terraform-dev \
--         --role-arn <redshift_analyst_role_arn> \
--         --file sql/redshift_streaming.sql --seccion validacion-analista --continue-on-error
--     Las dos últimas consultas DEBEN fallar con "permission denied".
-- ------------------------------------------------------------------------------
SELECT current_user AS usuario_bd;

-- El rol llega por el tag RedshiftDbRoles del rol IAM (no queda en svv_user_grants).
SELECT user_is_member_of(current_user, 'analytics_role')   AS tiene_analytics_role,
       user_is_member_of(current_user, 'stream_refresher') AS tiene_stream_refresher,
       has_schema_privilege('analytics', 'usage')          AS usa_analytics,
       has_schema_privilege('lakehouse', 'usage')          AS usa_lakehouse,
       has_schema_privilege('stream', 'usage')             AS usa_stream,
       has_schema_privilege('kds', 'usage')                AS usa_kds;

SELECT minuto, SUM(importe_compras) AS importe_compras
FROM analytics.mv_ventas_por_minuto
GROUP BY 1
ORDER BY 1 DESC
LIMIT 3;

SELECT COUNT(*) AS ventanas_iceberg
FROM lakehouse.ventanas_clientes;

SELECT cliente_id, compras_15m, ventanas_historicas, diagnostico
FROM analytics.v_clientes_stream_vs_lakehouse
ORDER BY cliente_id
LIMIT 3;

-- Debe fallar: el payload crudo no es accesible para analistas.
SELECT COUNT(*) FROM stream.mv_eventos_kinesis;

-- Debe fallar: no puede leer el stream de Kinesis directamente.
SELECT COUNT(*) FROM kds."data-platform-dev-events";


-- @seccion: reset
-- ------------------------------------------------------------------------------
-- 14. Limpieza (sólo a pedido: --seccion reset). Borra los objetos para volver
--     a ejecutar el script desde cero. Los datos se vuelven a leer del stream
--     (retención de 24 h) en el siguiente refresco.
-- ------------------------------------------------------------------------------
DROP PROCEDURE ops.sp_refrescar_stream();
DROP VIEW IF EXISTS analytics.v_clientes_stream_vs_lakehouse;
DROP VIEW IF EXISTS monitoring.v_lag_ingesta;
DROP VIEW IF EXISTS monitoring.v_scans_stream;
DROP VIEW IF EXISTS monitoring.v_refrescos;
DROP MATERIALIZED VIEW IF EXISTS analytics.mv_calidad_por_minuto;
DROP MATERIALIZED VIEW IF EXISTS analytics.mv_ventas_por_minuto;
DROP MATERIALIZED VIEW IF EXISTS stream.mv_eventos;
DROP MATERIALIZED VIEW IF EXISTS stream.mv_eventos_kinesis;
```

```
DROP SCHEMA IF EXISTS stream;
DROP SCHEMA IF EXISTS analytics;
DROP SCHEMA IF EXISTS monitoring;
DROP SCHEMA IF EXISTS ops;
DROP SCHEMA IF EXISTS lakehouse;
DROP SCHEMA IF EXISTS kds;
DROP ROLE analytics_role FORCE;
DROP ROLE stream_refresher FORCE;
```

6. Evidencia de ejecución

Corrida del 10/11/2026 (horas en UTC). Tráfico de prueba: `kinesis_producer.py` con 8–30 % de eventos con *schema drift* (JSON roto, campo faltante, importe de texto, timestamp con otro formato, campo nuevo). Los resultados son la salida real de `sql/redshift_streaming.sql`, secciones `validacion` y `validacion-analista`, ejecutadas por la Data API. Las capturas de consola están en §6.8.

6.1 Datos fluyendo de Kinesis a Redshift

Última tanda del productor (12 lotes de 20 eventos cada 10 s):

```
Lote 1/12: 20 eventos enviados

Lote 2/12: 20 eventos enviados

Lote 3/12: 20 eventos enviados

Lote 4/12: 20 eventos enviados

Lote 5/12: 20 eventos enviados

Lote 6/12: 20 eventos enviados

Lote 7/12: 20 eventos enviados

Lote 8/12: 20 eventos enviados

Lote 9/12: 20 eventos enviados

Lote 10/12: 20 eventos enviados

Lote 11/12: 20 eventos enviados

Lote 12/12: 20 eventos enviados

Proceso terminado. Registros enviados: 240

Stream: data-platform-dev-events

Claves de partición: 10 clientes diferentes

Eventos con schema drift: 14 {'json_roto': 2, 'sin_cliente': 3, 'importe_texto': 3, 'campo_nuevo': 4,
'timestamp_invalido': 2}
```

Bronze: registros por shard y validez del JSON (incluye las tandas anteriores):

```
SELECT shard_id, is_valid_json, COUNT(*) AS registros,
```

shard_id	is_valid_json	registros	primera_llegada	ultima_llegada
shardId-000000000000	False	4	2026-11-10 16:27:23.983	2026-11-10 16:36:36.10
shardId-000000000000	True	280	2026-11-10 16:16:22.767	2026-11-10 16:37:58.064
shardId-000000000001	False	12	2026-11-10 16:16:22.767	2026-11-10 16:32:39.343
shardId-000000000001	True	414	2026-11-10 16:16:22.764	2026-11-10 16:37:58.064

(4 filas) · OK en 0.27 s (filas afectadas: 4)

Silver: campos JSON convertidos a tipos nativos, sin errores de casting:

```
SELECT event_id, event_type, cliente_id, importe, event_ts, llegada_kinesis_ts, ingesta_redshift_ts
```

event_id	event_type	cliente_id	importe	event_ts	llegada_kinesis_ts	ingesta_redshift_ts
240	compra	cliente-010	308.56	2026-11-10 16:37:56.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
239	consulta	cliente-009	54.02	2026-11-10 16:37:57.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
238	compra	cliente-008	166.51	2026-11-10 16:37:59.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
237	consulta	cliente-007	365.80	2026-11-10 16:37:58.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
236	compra	cliente-006	342.04	2026-11-10 16:37:56.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
235	consulta	cliente-005	46.73	2026-11-10 16:37:55.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
234	compra	cliente-004	278.08	2026-11-10 16:37:59.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
233	consulta	cliente-003	80.67	2026-11-10 16:37:57.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
232	compra	cliente-002	136.70	2026-11-10 16:37:59.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05
231	consulta	cliente-001	58.72	2026-11-10 16:38:00.21478	2026-11-10 16:37:58.064	2026-11-10 16:38:05

(10 filas) · OK en 0.17 s (filas afectadas: 10)

```
SELECT column_name, data_type, character_maximum_length, numeric_precision, numeric_scale
```

column_name	data_type	character_maximum_length	numeric_precision	numeric_scale
shard_id	character	20	NULL	NULL
sequence_number	character varying	128	NULL	NULL
llegada_kinesis_ts	timestamp without time zone	NULL	NULL	NULL
ingesta_redshift_ts	timestamp without time zone	NULL	NULL	NULL
event_id	bigint	NULL	64	0
event_type	character varying	48	NULL	NULL
cliente_id	character varying	64	NULL	NULL
importe	numeric	NULL	12	2
event_ts	timestamp without time zone	NULL	NULL	NULL
es_valido	boolean	NULL	NULL	NULL
motivo_invalido	character varying	26	NULL	NULL
payload	super	NULL	NULL	NULL

(12 filas) · OK en 0.13 s (filas afectadas: 12)

6.2 Schema drift controlado

Los registros que no cumplen el contrato no detienen la ingesta: se conservan con su payload original y se clasifican. El evento con un campo nuevo (`canal`) es válido, porque se ignora el campo adicional.

```
SELECT motivo_invalido, COUNT(*) AS registros,
```

motivo_invalido	registros	ejemplo
campo_obligatorio_faltante	19	{"event_id":15,"event_type":"consulta","importe":261.96,"timestamp":"2026-11-10T16:16:25.282661+00:00"}
json_invalido	16	null
importe_no_numerico	15	{"event_id":123,"event_type":"consulta","cliente_id":"cliente-003","importe":"N/A","timestamp":"2026-11-10T16:30:53.036939+00:00"}
timestamp_invalido	14	{"event_id":104,"event_type":"compra","cliente_id":"cliente-004","importe":316.24,"timestamp":"10/11/2026 25:61"}

(4 filas) · OK en 0.17 s (filas afectadas: 4)

```
SELECT minuto, resultado, registros
```

minuto	resultado	registros
2026-11-10 16:37:00	ok	96
2026-11-10 16:37:00	campo_obligatorio_faltante	2
2026-11-10 16:37:00	timestamp_invalido	1
2026-11-10 16:37:00	importe_no_numerico	1
2026-11-10 16:36:00	ok	116
2026-11-10 16:36:00	importe_no_numerico	2
2026-11-10 16:36:00	timestamp_invalido	1
2026-11-10 16:36:00	json_invalido	1
2026-11-10 16:35:00	ok	18
2026-11-10 16:35:00	campo_obligatorio_faltante	1
2026-11-10 16:35:00	json_invalido	1
2026-11-10 16:33:00	ok	19
2026-11-10 16:33:00	json_invalido	1
2026-11-10 16:32:00	ok	88
2026-11-10 16:32:00	importe_no_numerico	5

(15 filas) · OK en 0.15 s (filas afectadas: 15)

6.3 Métricas de tiempo real (gold)

```
SELECT minuto, SUM(operaciones) AS operaciones, SUM(compras) AS compras,
```

minuto	operaciones	compras	importe_compras	compra_maxima
2026-11-10 16:38:00	1	0	0.00	NULL
2026-11-10 16:37:00	98	50	12264.29	489.43
2026-11-10 16:36:00	103	51	14506.72	488.40
2026-11-10 16:35:00	28	14	3722.61	496.95
2026-11-10 16:33:00	13	6	1575.73	433.03
2026-11-10 16:32:00	94	50	12987.73	483.64
2026-11-10 16:31:00	92	47	12649.03	495.47
2026-11-10 16:30:00	96	48	11911.57	481.44
2026-11-10 16:29:00	37	19	6003.17	499.61
2026-11-10 16:27:00	50	26	6759.99	478.78

(10 filas) · OK en 0.22 s (filas afectadas: 10)

6.4 Integración Lakehouse: stream e Iceberg en la misma sesión SQL

Ritmo de compras de los últimos 15 minutos (stream) frente al promedio histórico por ventana (Iceberg). Los clientes impares sólo generan eventos `consulta`, por eso no tienen compras.

```
SELECT *
```

cliente_id	minutos_activos	compras_15m	importe_15m	importe_por_minuto	ventanas_historicas	importe_promedio_ventana	ratio_vs_historico	alertas_historicas	ultima_ventana_historica	ultimo_minuto	diagnostico
cliente-001	10	0	0.00	0.00	6	0.00	NULL	0	2026-11-01 22:14:00	2026-11-10 16:38:00	SIN_COMPRAS
cliente-002	9	61	16682.42	1853.60	5	2411.32	0.77	1	2026-11-01 22:14:00	2026-11-10 16:37:00	NORMAL
cliente-003	9	0	0.00	0.00	6	0.00	NULL	0	2026-11-01 22:14:00	2026-11-10 16:37:00	SIN_COMPRAS
cliente-004	8	60	16367.79	2045.97	6	2424.42	0.84	1	2026-11-01 22:14:00	2026-11-10 16:37:00	NORMAL
cliente-005	9	0	0.00	0.00	6	0.00	NULL	0	2026-11-01 22:14:00	2026-11-10 16:37:00	SIN_COMPRAS
cliente-006	9	63	17861.02	1984.55	6	2058.50	0.96	1	2026-11-01 22:14:00	2026-11-10 16:37:00	NORMAL
cliente-007	8	0	0.00	0.00	6	0.00	NULL	0	2026-11-01 22:14:00	2026-11-10 16:37:00	SIN_COMPRAS
cliente-008	9	63	15439.12	1715.45	6	2016.18	0.85	1	2026-11-01 22:14:00	2026-11-10 16:37:00	NORMAL
cliente-009	9	0	0.00	0.00	6	0.00	NULL	0	2026-11-01 22:14:00	2026-11-10 16:37:00	SIN_COMPRAS
cliente-010	9	64	16030.49	1781.16	6	2258.82	0.79	1	2026-11-01 22:14:00	2026-11-10 16:37:00	NORMAL

(10 filas) · OK en 1.31 s (filas afectadas: 10)

JOIN ad hoc entre la última hora del stream y el estado acumulado que Flink persistió en Iceberg:

```
SELECT s.cliente_id,
```

cliente_id	compras_stream_ultima_hora	importe_stream_ultima_hora	ventanas_flink	importe_acumulado_iceberg	importe_total_cliente	ultima_ventana_iceberg
cliente-001	0	0.00	10	0.00	0.00	2026-11-01 22:14:00
cliente-002	66	17890.89	9	21597.10	39487.99	2026-11-01 22:14:00
cliente-003	0	0.00	9	0.00	0.00	2026-11-01 22:14:00
cliente-004	64	17322.87	9	23150.39	40473.26	2026-11-01 22:14:00
cliente-005	0	0.00	9	0.00	0.00	2026-11-01 22:14:00
cliente-006	67	18895.69	10	19435.42	38331.11	2026-11-01 22:14:00
cliente-007	0	0.00	10	0.00	0.00	2026-11-01 22:14:00
cliente-008	65	16208.59	10	21186.50	37395.09	2026-11-01 22:14:00
cliente-009	0	0.00	10	0.00	0.00	2026-11-01 22:14:00
cliente-010	69	17284.52	10	23677.08	40961.60	2026-11-01 22:14:00

(10 filas) · OK en 1.18 s (filas afectadas: 10)

6.5 Metadatos de Iceberg

Desde Redshift: tabla externa, esquema y forma en que el motor la leyó (formato Iceberg, archivos resueltos desde los manifests).

```
SELECT schemaname, tablename, location,
```

schemaname	tablename	location	ultimo_commit_iceberg	parametros_glue
lakehouse	ventanas_clientes	s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes	2026-11-01 21:52:17	{"commit.retry.num-retries":"10","iceberg.table.lastUpdatedMs":"1793569937146","iceberg.table.uuid":"7c5872cc-65ca-4195-a1cb-660

(1 filas) · OK en 0.15 s (filas afectadas: 1)

```
SELECT columnnum, columnname, external_type, part_key
```

columnnum	columnname	external_type	part_key
1	window_start	timestamp	0
2	window_end	timestamp	0
3	cliente_id	string	0
4	total_operaciones	bigint	0
5	cantidad_compras	bigint	0
6	importe_total_compras	double	0
7	importe_promedio_compras	double	0
8	ventanas_procesadas	bigint	0
9	importe_acumulado	double	0
10	variacion_pct	double	0
11	alerta	string	0

(11 filas) · OK en 0.17 s (filas afectadas: 11)

```
SELECT start_time, TRIM(table_name) AS tabla, TRIM(file_format) AS formato,
```

start_time	tabla	formato	scanned_files	returned_rows	returned_bytes	duracion_us
2026-11-10 16:38:24.458162	analytics.lakehouse.ventanas_clientes	Iceberg	10	59	2301	88606
2026-11-10 16:38:22.407932	analytics.lakehouse.ventanas_clientes	Iceberg	10	59	2125	96968
2026-11-10 16:34:27.707647	analytics.lakehouse.ventanas_clientes	Iceberg	10	59	1357	132077
2026-11-10 16:34:25.03926	analytics.lakehouse.ventanas_clientes	Iceberg	10	59	0	43958
2026-11-10 16:34:13.499976	analytics.lakehouse.ventanas_clientes	Iceberg	10	59	2301	9879

(5 filas) · OK en 1.12 s (filas afectadas: 5)

Desde Athena y Glue: historial de snapshots (uno por checkpoint de Flink), particiones, archivos y puntero `metadata_location`.

```
Athena: SELECT committed_at, snapshot_id, parent_id, operation, summary['added-records'] AS added_records,
summary['total-records'] AS total_records FROM "ventanas_clientes$snapshots" ORDER BY committed_at
```

committed_at	snapshot_id	parent_id	operation	added_records	total_records
2026-11-01 22:11:25.639 UTC	3077757530987269976	NULL	append	19	19
2026-11-01 22:12:23.628 UTC	4210774569761478939	3077757530987269976	append	10	29
2026-11-01 22:13:23.598 UTC	6388884917182056901	4210774569761478939	append	10	39
2026-11-01 22:14:23.782 UTC	3876041861386617884	6388884917182056901	append	10	49
2026-11-01 22:15:23.592 UTC	500829894777509235	3876041861386617884	append	10	59

Athena: SELECT made_current_at, snapshot_id, is_current_ancestor FROM "ventanas_clientes$history" ORDER BY made_current_at

made_current_at	snapshot_id	is_current_ancestor
2026-11-01 22:11:25.639 UTC	3077757530987269976	true
2026-11-01 22:12:23.628 UTC	4210774569761478939	true
2026-11-01 22:13:23.598 UTC	6388884917182056901	true
2026-11-01 22:14:23.782 UTC	3876041861386617884	true
2026-11-01 22:15:23.592 UTC	500829894777509235	true

Athena: SELECT partition, record_count, file_count, total_size FROM "ventanas_clientes$partitions"

partition	record_count	file_count	total_size
{window_start_day=2026-11-01}	59	10	39574

Athena: SELECT file_path, record_count, file_size_in_bytes FROM "ventanas_clientes$files" LIMIT 5

file_path	record_count	file_size_in_bytes
s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes/data/window_start_day=2026-11-01/00000-0-9d6449e4-cd2a-4a52-a747-40bd2e35a98e-00004.parquet	8	4016
s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes/data/window_start_day=2026-11-01/00001-0-232ae620-b24a-47cb-a5b6-e6df7630629e-00004.parquet	2	3724
s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes/data/window_start_day=2026-11-01/00000-0-9d6449e4-cd2a-4a52-a747-40bd2e35a98e-00002.parquet	8	4162
s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes/data/window_start_day=2026-11-01/00001-0-232ae620-b24a-47cb-a5b6-e6df7630629e-00002.parquet	2	3885
s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes/data/window_start_day=2026-11-01/00000-0-9d6449e4-cd2a-4a52-a747-40bd2e35a98e-00001.parquet	15	4191

Glue get-table


```
{
    "tipo": "ICEBERG",
    "metadata_location": "s3://data-platform-dev-659500704179-
bucket/lakehouse/lakehouse_db.db/ventanas_clientes/metadata/00005-65e5328a-b81b-47a8-b0ee-
51d5f1ed4aa7.metadata.json",
    "previous": "s3://data-platform-dev-659500704179-
bucket/lakehouse/lakehouse_db.db/ventanas_clientes/metadata/00004-cc5f3d40-cc00-48e9-bc97-
2fcb98cb6176.metadata.json",
    "location": "s3://data-platform-dev-659500704179-bucket/lakehouse/lakehouse_db.db/ventanas_clientes"
}
```

S3 metadata/ (aws s3 ls muestra hora local UTC-3)

```
2026-11-01 18:52:18       1913 00000-d0ef9405-69bc-4a41-8621-eb30c4632bd5.metadata.json
2026-11-01 19:11:26       4152 00001-b9a6329f-7917-4680-a2cb-6266a7f9c6ae.metadata.json
2026-11-01 19:12:24       5561 00002-b959cc8d-528c-4c55-8777-2010e03c49e1.metadata.json
2026-11-01 19:13:24       6970 00003-53cf891a-c2dc-45e1-8cd8-ebb9d97d1c5a.metadata.json
2026-11-01 19:14:24       8379 00004-cc5f3d40-cc00-48e9-bc97-2fcb98cb6176.metadata.json
2026-11-01 19:15:24       9784 00005-65e5328a-b81b-47a8-b0ee-51d5f1ed4aa7.metadata.json
2026-11-01 19:11:26       8259 0a1516d6-b27c-4f59-9879-9edf0db25fa0-m0.avro
2026-11-01 19:14:24       8228 140256d4-c95f-431a-8907-dfd4dc8da89d-m0.avro
2026-11-01 19:15:24       8232 20895026-67bb-4f42-b7f9-92dc66a49f7a-m0.avro
2026-11-01 19:13:24       8248 6d145414-9fa9-43e3-a53c-7d17796e23f7-m0.avro
2026-11-01 19:12:24       8238 d9926698-3213-49fb-97b0-573bbb7a433c-m0.avro
2026-11-01 19:11:26       4496 snap-3077757530987269976-1-0a1516d6-b27c-4f59-9879-9edf0db25fa0.avro
2026-11-01 19:14:24       4664 snap-3876041861386617884-1-140256d4-c95f-431a-8907-dfd4dc8da89d.avro
2026-11-01 19:12:24       4575 snap-4210774569761478939-1-d9926698-3213-49fb-97b0-573bbb7a433c.avro
2026-11-01 19:15:24       4711 snap-500829894777509235-1-20895026-67bb-4f42-b7f9-92dc66a49f7a.avro
2026-11-01 19:13:24       4621 snap-6388884917182056901-1-6d145414-9fa9-43e3-a53c-7d17796e23f7.avro
```

6.6 Seguridad: acceso restringido al rol de analítica

Sesión abierta asumiendo el rol IAM `data-platform-dev-redshift-analyst`, con credenciales temporales y sin contraseña de base de datos. Redshift crea el usuario `IAMR:<rol>` y le otorga `analytics_role` a partir del tag. Puede leer analytics, el lakehouse y el JOIN; el dato crudo (`stream`) y el stream (`kds`) le devuelven *permission denied*.

```
SELECT current_user AS usuario_bd
```

usuario_bd
IAMR:data-platform-dev-redshift-analyst

(1 filas) · OK en 0.03 s (filas afectadas: 1)

```
SELECT user_is_member_of(current_user, 'analytics_role') AS tiene_analytics_role,
```

tiene_analytics_role	tiene_stream_refresher	usa_analytics	usa_lakehouse	usa_stream	usa_kds
True	False	True	True	False	False

(1 filas) · OK en 0.03 s (filas afectadas: 1)

```
SELECT COUNT(*) AS ventanas_iceberg
```

ventanas_iceberg
59

(1 filas) · OK en 0.56 s (filas afectadas: 1)

```
SELECT cliente_id, compras_15m, ventanas_historicas, diagnostico
```

cliente_id	compras_15m	ventanas_historicas	diagnostico
cliente-001	0	6	SIN_COMPRAS
cliente-002	61	5	NORMAL
cliente-003	0	6	SIN_COMPRAS

(3 filas) · OK en 1.26 s (filas afectadas: 3)

```
SELECT COUNT(*) FROM stream.mv_eventos_kinesis
```

ERROR (FAILED): ERROR: permission denied for schema stream

```
SELECT COUNT(*) FROM kds."data-platform-dev-events"
```

2 sentencias con error

ERROR (FAILED): ERROR: permission denied for schema kds

Refresco programado: lo ejecuta el usuario IAMR del rol del scheduler, cada 60 s, sin secreto.

```
SELECT TRIM(u.usename) AS usuario_bd,
```

usuario_bd	sentencia	estado	start_time	duracion_ms
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:38:11.57476	3079
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:37:11.530823	3850
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:36:11.435586	3950
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:35:11.448125	3160
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:34:11.445946	3706
IAMR:data-platform-dev-redshift-refresh	CALL ops.sp_refrescar_stream()	success	2026-11-10 16:33:11.386567	4467

(6 filas) · OK en 0.96 s (filas afectadas: 6)

6.7 Monitoreo: lag de ingesta y refrescos

`lag_kinesis_a_redshift` mide el tiempo desde que Kinesis aceptó el registro hasta que un REFRESH lo materializó. Con la cadencia de 60 s, el promedio está entre 20 y 30 s y el máximo cerca de 60 s, lo esperado. El valor de 593 s del minuto 16:16 corresponde a datos enviados antes de crear la vista. Los valores negativos de `lag_evento_a_kinesis` son diferencias de reloj de 1–3 s entre la PC del productor y AWS.

```
SELECT *
```

minuto	registros	lag_kinesis_a_redshift_prom_s	lag_kinesis_a_redshift_max_s	lag_evento_a_kinesis_prom_s	ultimo_refresco_ts
2026-11-10 16:37:00	100	19.5	42.0	0.4	2026-11-10 16:38:05
2026-11-10 16:36:00	120	30.7	58.3	0.6	2026-11-10 16:37:11
2026-11-10 16:35:00	20	21.5	21.5	-0.8	2026-11-10 16:36:11
2026-11-10 16:33:00	20	8.1	8.1	0.7	2026-11-10 16:33:11
2026-11-10 16:32:00	100	31.2	54.8	0.3	2026-11-10 16:33:11
2026-11-10 16:31:00	100	29.2	52.8	0.4	2026-11-10 16:32:11
2026-11-10 16:30:00	100	28.3	50.8	0.0	2026-11-10 16:31:11
2026-11-10 16:29:00	40	24.9	30.6	0.2	2026-11-10 16:30:16
2026-11-10 16:27:00	60	5.0	7.0	-2.8	2026-11-10 16:27:54
2026-11-10 16:16:00	50	593.1	594.2	-3.1	2026-11-10 16:26:17

(10 filas) · OK en 0.44 s (filas afectadas: 10)

SELECT *

scan_ts	mv_name	shard_id	scanned_rows	skipped_rows	scanned_bytes	lag_from_latest	min_latency_s	max_latency_s	stream_record_time_min	stream_record_time_max
2026-11-10 16:38:11.915292	mv_eventos_kinesis	shardId-000000000000	0	0	0	0	NULL	NULL	2000-01-01 00:00:00	2000-01-01 00:00:00
2026-11-10 16:38:11.887683	mv_eventos_kinesis	shardId-000000000001	0	0	0	0	NULL	NULL	2000-01-01 00:00:00	2000-01-01 00:00:00
2026-11-10 16:38:05.593937	mv_eventos_kinesis	shardId-000000000001	48	0	10276	0	7	43	2026-11-10 16:37:22.96	2026-11-10 16:37:58.064
2026-11-10 16:38:05.503038	mv_eventos_kinesis	shardId-000000000000	32	0	6847	0	7	43	2026-11-10 16:37:22.963	2026-11-10 16:37:58.064
2026-11-10 16:37:11.863258	mv_eventos_kinesis	shardId-000000000001	72	0	15466	0	0	59	2026-11-10 16:36:12.722	2026-11-10 16:37:11.149
2026-11-10 16:37:11.860958	mv_eventos_kinesis	shardId-000000000000	48	0	10173	0	0	59	2026-11-10 16:36:12.725	2026-11-10 16:37:11.149
2026-11-10 16:36:11.77051	mv_eventos_kinesis	shardId-000000000001	24	0	5148	0	10	22	2026-11-10 16:35:49.504	2026-11-10 16:36:01.055
2026-11-10 16:36:11.763426	mv_eventos_kinesis	shardId-000000000000	16	0	3294	0	10	22	2026-11-10 16:35:49.506	2026-11-10 16:36:01.055

(8 filas) · OK en 2.01 s (filas afectadas: 8)

SELECT *

vista	start_time	duracion_ms	status	refresh_type
analytics.mv_calidad_por_minuto	2026-11-10 16:38:14.130802	335	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
analytics.mv_ventas_por_minuto	2026-11-10 16:38:13.640967	487	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
stream.mv_eventos	2026-11-10 16:38:13.225419	260	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
stream.mv_eventos_kinesis	2026-11-10 16:38:11.579892	1456	Refresh successfully updated MV incrementally. Stream returned no new data	Manual
analytics.mv_calidad_por_minuto	2026-11-10 16:38:08.751663	708	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
analytics.mv_ventas_por_minuto	2026-11-10 16:38:07.742821	1006	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
stream.mv_eventos	2026-11-10 16:38:06.838645	376	Refresh successfully updated MV incrementally, but MV depends on an MV that is not up to date.	Manual
stream.mv_eventos_kinesis	2026-11-10 16:38:05.19538	1471	Refresh successfully updated MV incrementally	Manual

(8 filas) · OK en 1.58 s (filas afectadas: 8)

SELECT schema_name, name, autorefresh, is_stale, state

schema_name	name	autorefresh	is_stale	state
analytics	mv_calidad_por_minuto	f	f	1
analytics	mv_ventas_por_minuto	f	f	1
stream	mv_eventos	f	f	1
stream	mv_eventos_kinesis	f	t	1

(4 filas) · OK en 0.55 s (filas afectadas: 4)

SELECT record_time, mv_name, partition_id, error_code, error_reason

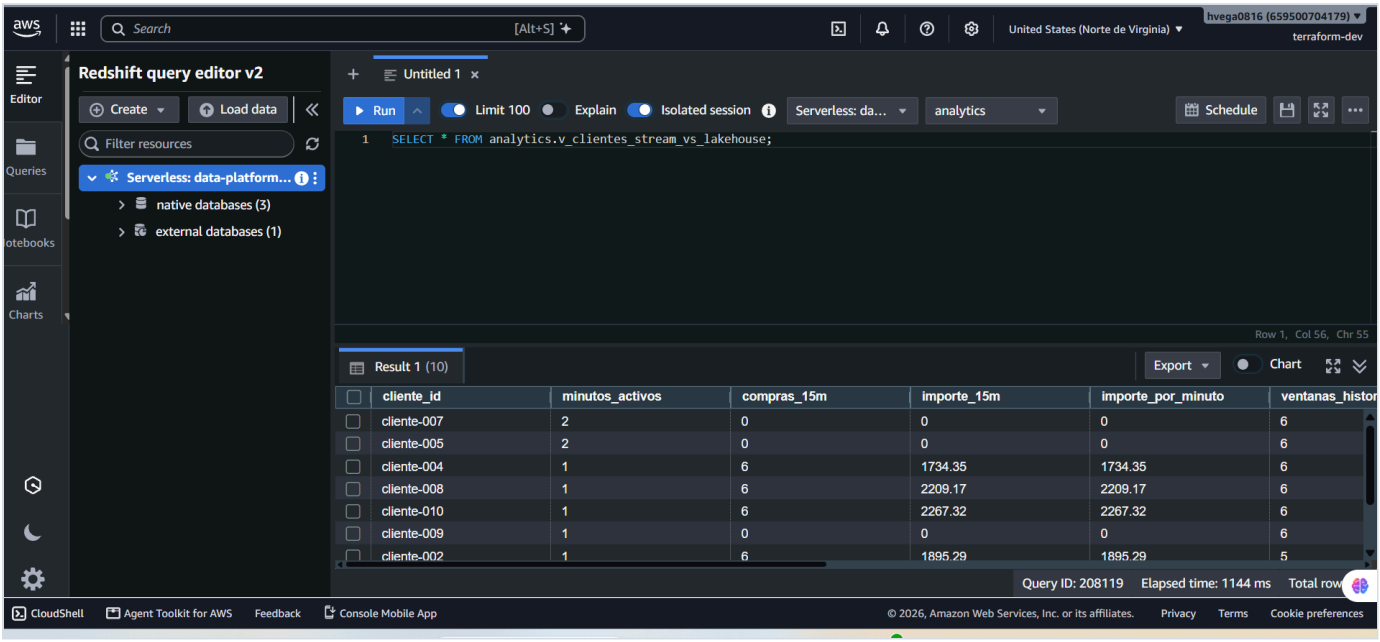
record_time	mv_name	partition_id	error_code	error_reason

(0 filas) · OK en 2.08 s (filas afectadas: 0)

6.8 Capturas de consola

Captura pendiente: `docs/evidencias-redshift/consola-redshift-mv-stream.png`

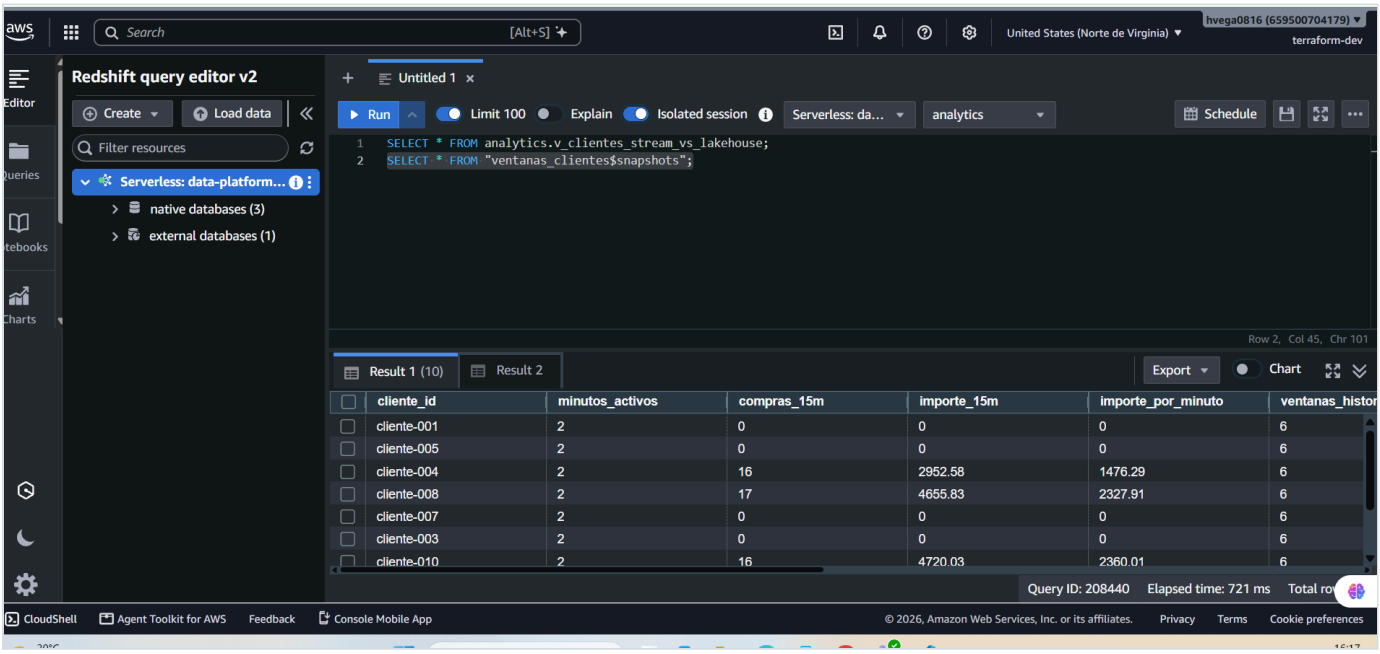
Query Editor v2: SELECT sobre stream.mv_eventos con datos recién llegados de Kinesis



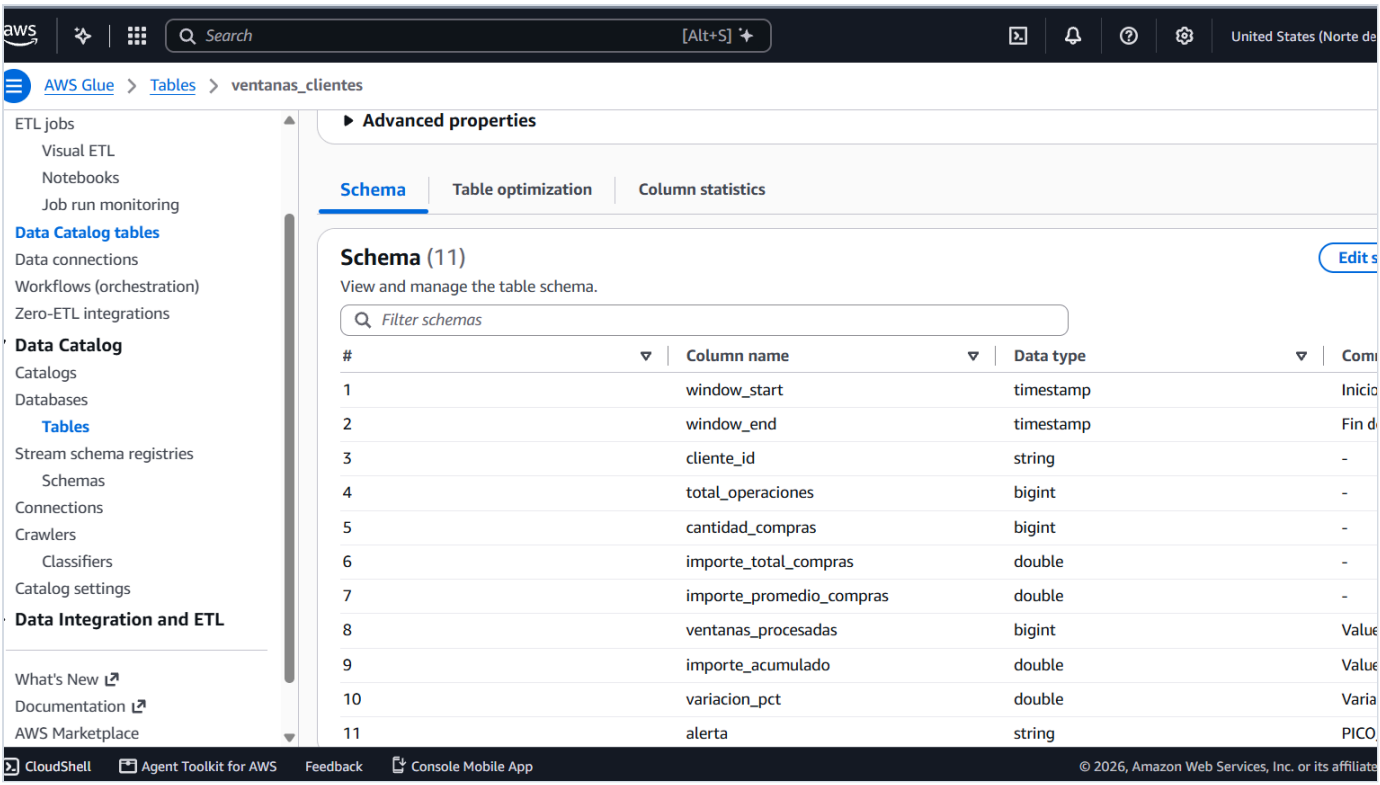
Query Editor v2: JOIN analytics.v_clientes_stream_vs_lakehouse (stream + Iceberg)

Captura pendiente: `docs/evidencias-redshift/consola-redshift-workgroup.png`

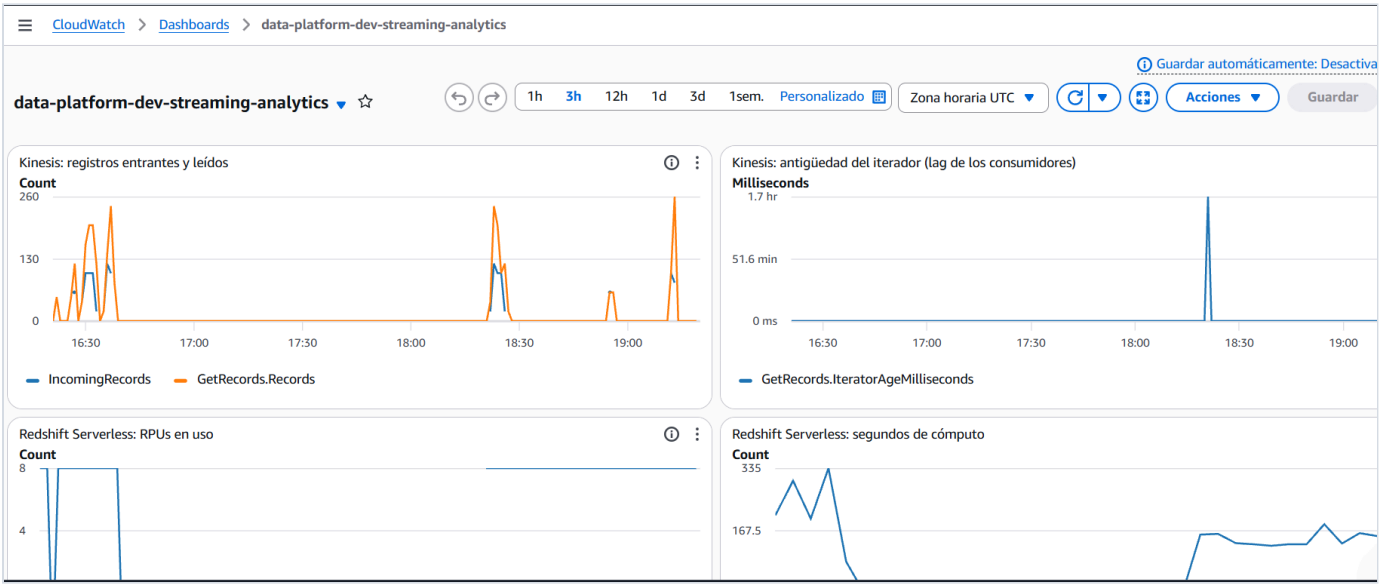
Redshift Serverless: workgroup privado y rol IAM asociado al namespace



Athena: metadatos Iceberg ventanas_clientes$snapshots



Glue Data Catalog: tabla Iceberg ventanas_clientes



CloudWatch: dashboard data-platform-dev-streaming-analytics

7. Estrategia de refresco y costos (latency trade-off)

Alternativa	Latencia	Efecto	Decisión
Refresco cada 1 s	~1 s	Un REFRESH de la cadena tarda 3–4,5 s, así que se solaparían y saturarían el workgroup. Además, una métrica por minuto no gana precisión.	Descartada
AUTO REFRESH YES	Variable	Redshift no lo admite en vistas anidadas; sobre el stream no permite fijar la cadencia y en Serverless deja el cómputo activo todo el tiempo	Descartada
Scheduler cada 60 s + REFRESH incremental	20–30 s promedio, ≤ 60 s	Cada ciclo procesa sólo lo nuevo, tarda 3–4,5 s y respeta la granularidad por minuto de las métricas	**Elegida**

	máximo		
Manual / bajo demanda	La de la consulta	Sin costo en reposo; adecuada para entornos de desarrollo	Fuera de las pruebas

Costos: Redshift Serverless cobra por RPU-segundo sólo mientras hay consultas, con 8 RPU ≈ US$ 3/h activo. Con el refresco cada minuto el workgroup queda activo casi todo el tiempo, por eso el scheduler está apagado por defecto (`redshift_refresh_schedule_enabled = false`). Además, el *usage limit* lo desactiva al superar 16 RPU-hora por día. Si el refresco está apagado más de 24 h (la retención del stream), los registros más viejos se pierden para Redshift. El histórico sigue disponible en S3 (Firehose) e Iceberg (Flink).

8. Cómo reproducir

```
cd infra/environments/dev
terraform init
terraform apply                     # módulo redshift (+ subred en la 3.ª AZ)
SECRET=$(terraform output -raw redshift_admin_secret_arn)
ANALYST=$(terraform output -raw redshift_analyst_role_arn)
cd ../../..

# 1. Esquemas externos, vistas materializadas, roles y permisos (una vez)
python scripts/redshift_sql.py --secret-arn $SECRET --file sql/redshift_streaming.sql

# 2. Refresco cada minuto: redshift_refresh_schedule_enabled = true en terraform.tfvars y terraform apply

# 3. Tráfico de prueba con schema drift
python scripts/kinesis_producer.py --lotes 12 --registros 20 --pausa 10 --max-retraso 5 --anomalias 8

# 4. Validación (administrador) y permisos (rol de analistas)
python scripts/redshift_sql.py --secret-arn $SECRET --file sql/redshift_streaming.sql --seccion validacion
python scripts/redshift_sql.py --profile terraform-dev --role-arn $ANALYST \
       --file sql/redshift_streaming.sql --seccion validacion-analista --continue-on-error

# 5. Este documento
python scripts/generar_informe_pdf.py
```